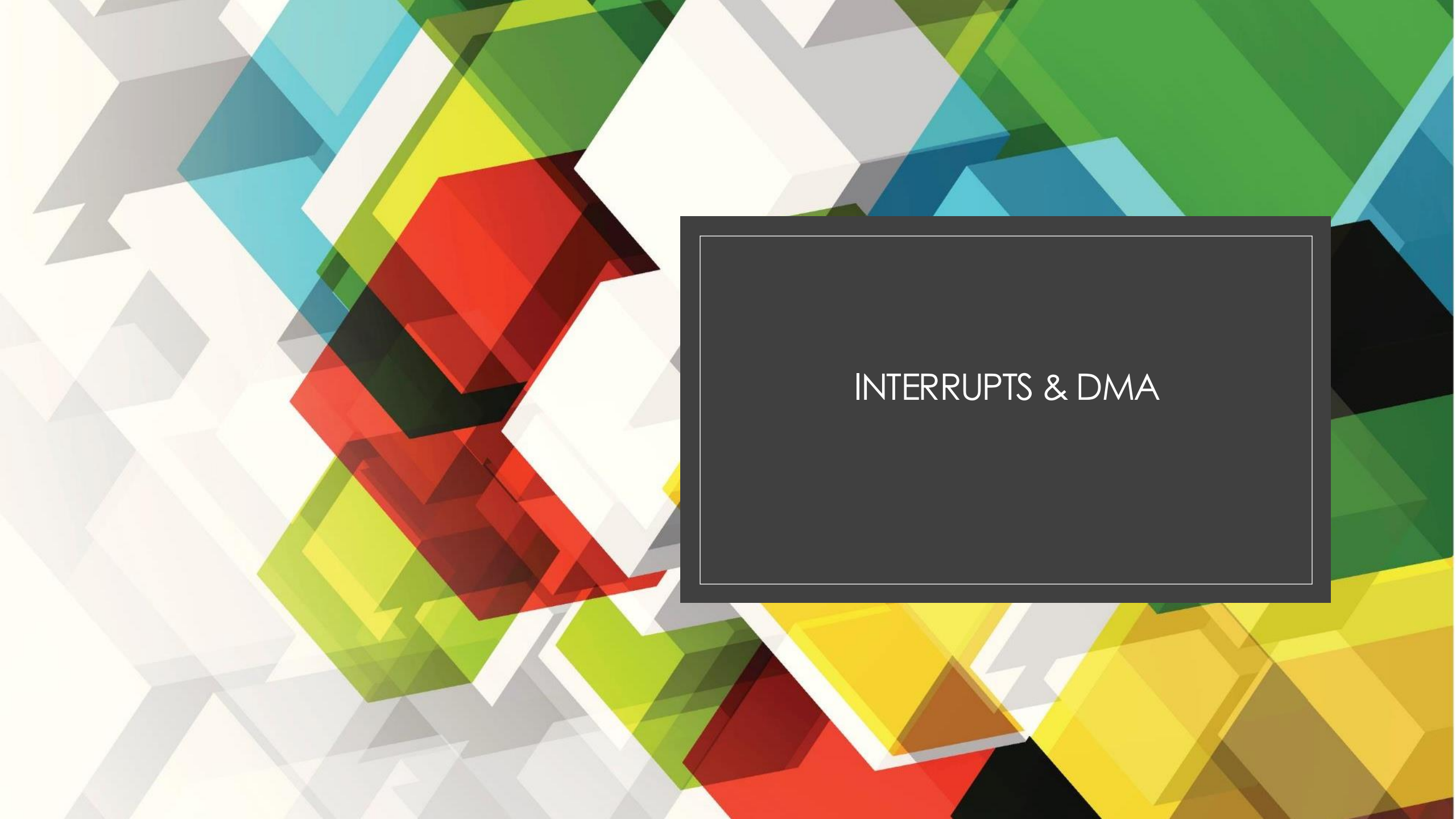




INTERRUPTS & DMA

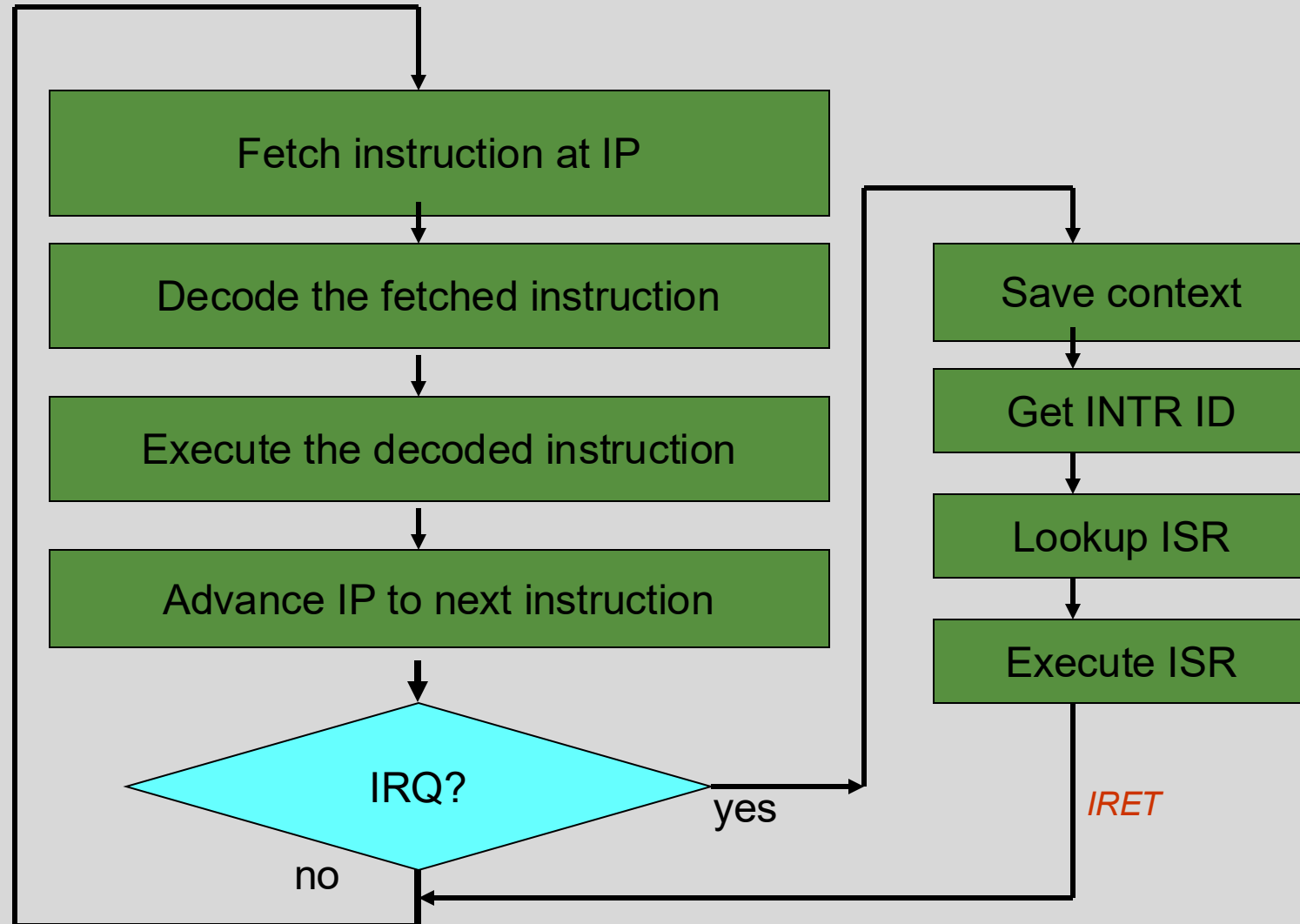
Interrupts

- **Polling (Inefficient for Infrequent I/O)**
- Takes ~3 instruction cycles:
 - Read status register
 - Check status bit
 - Branch if done
- CPU **constantly checks device**, wasting cycles
- Not ideal when I/O is **infrequent or slow**
- **Interrupt-Driven I/O (More Efficient)**
- **I/O device sends interrupt request (IRQ)** to CPU
- CPU checks for interrupts **after each instruction**
- If interrupt is present:
 - **Interrupt Handler (ISR)** is executed
 - Handles the device and resumes CPU tasks
- **CPU stays productive** instead of waiting

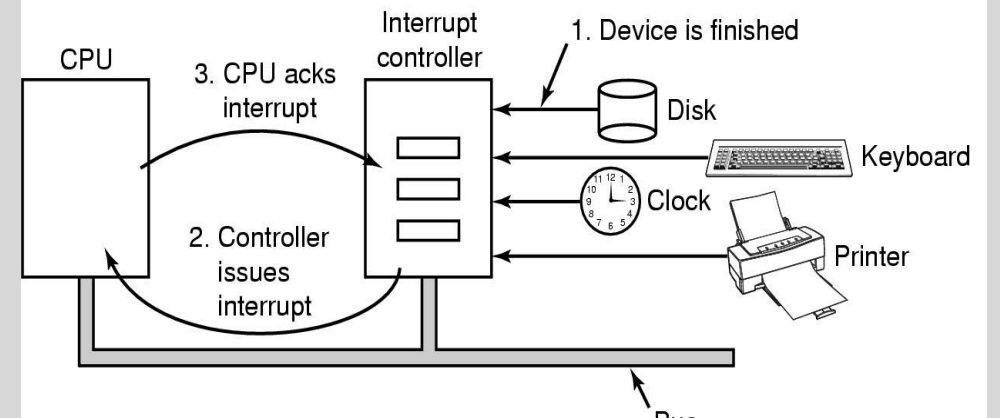
Maskable Interrupts

- CPU can **enable/disable** certain interrupts
 - Uses an **Interrupt Enable (IE)** flag or control bit
- Useful for **critical sections** of code
 - Prevents low-priority interruptions
- **Non-maskable interrupts (NMIs):**
 - **Always handled**, used for critical events (e.g. hardware failure)

Interrupt Service Routine



Interrupt Driven I/O



- **I/O Module Collects Data from Peripheral**

- The I/O module **interacts directly** with the external device (e.g., keyboard, disk)
- Meanwhile, the **CPU continues executing other instructions**: no time wasted waiting

- **I/O Module Interrupts CPU When Data is Ready**

- Once the data is available, the I/O module sends an **interrupt signal** to the CPU
- This notifies the CPU that the data can now be processed

- **CPU Acknowledges Interrupt and Requests Data**

- The CPU pauses its current task
- It sends a **read request** to the I/O module to retrieve the data

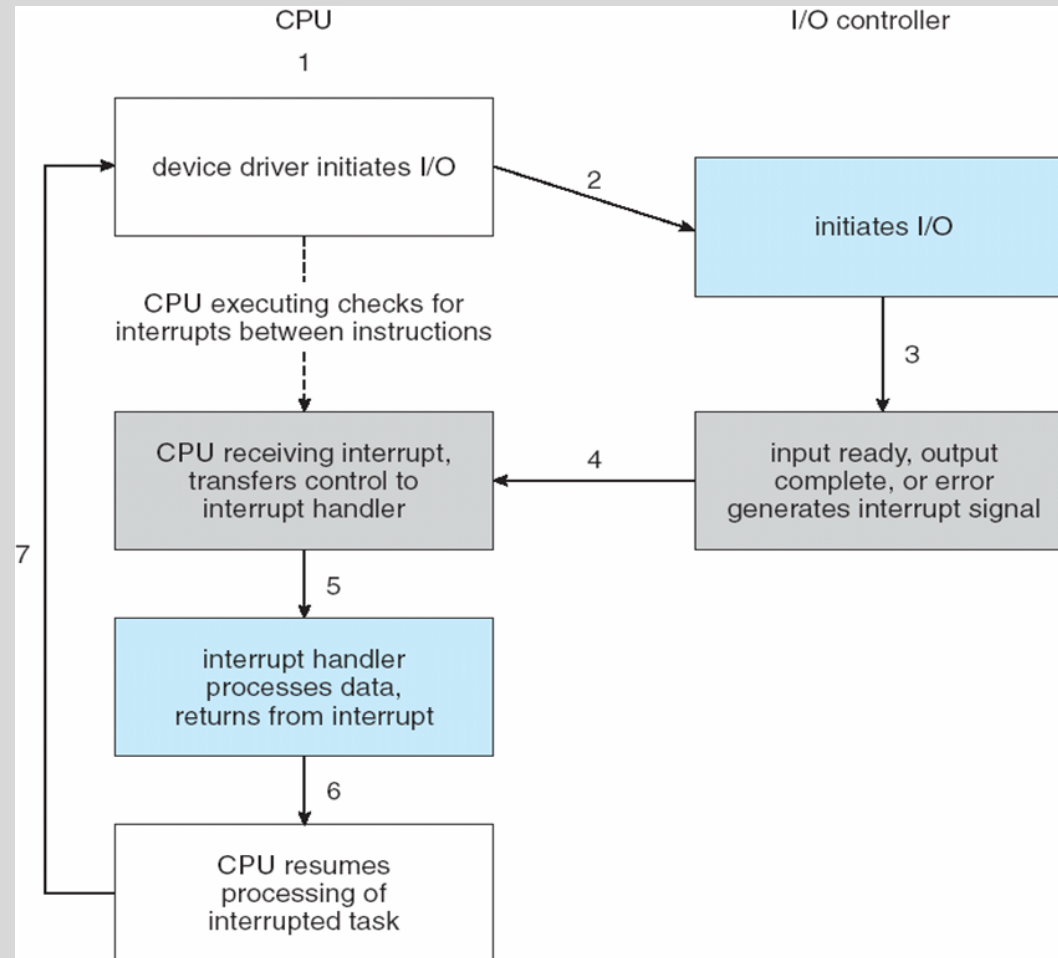
- **I/O Module Transfers Data to CPU**

- Data is moved from the I/O module to the CPU (via memory or registers)
- After the transfer, the CPU **resumes normal execution**

Interrupt Driven I/O

- I/O device issues an interrupt to indicate that it needs attention of CPU
- Interrupts are special signals initialed by I/O devices to catch the attention of the processor.
- Overcomes CPU waiting
- No repeated CPU checking of device
- An I/O interrupt is **asynchronous** w.r.t. instruction execution
- Is not associated with any instruction so doesn't prevent any instruction from completing

Interrupt-Driven I/O Cycle



Interrupt Driven I/O

- Advantages

- Relieves the processor from having to continuously poll for an I/O event; user program progress is only suspended during the actual transfer of I/O data to/from user memory space

Disadvantages

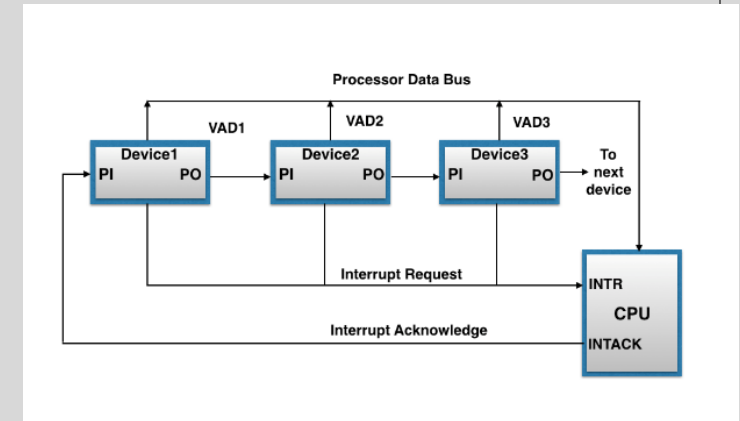
- Special hardware is needed to indicate the I/O device causing the interrupt and to save the necessary information prior to servicing the interrupt and to resume normal processing after servicing the interrupt

Challenges in Interrupt Driven I/O

- How do you identify the module issuing the interrupt?
 - Need a way to identify the device generating the interrupt
- How do you deal with multiple interrupts?
 - Can have different urgencies (so need a way to prioritize them)

Identifying Interrupting Module

- CPU asks each module in turn (Slow) Daisy Chain or Hardware poll
 - Interrupt Acknowledge sent down a chain
 - Module responsible places vector on bus
 - CPU uses vector to identify handler routine
- Vectored Interrupt
 - **Interrupt vector** to dispatch interrupt to correct handler



Ex: Intel Pentium Processor Event-Vector Table

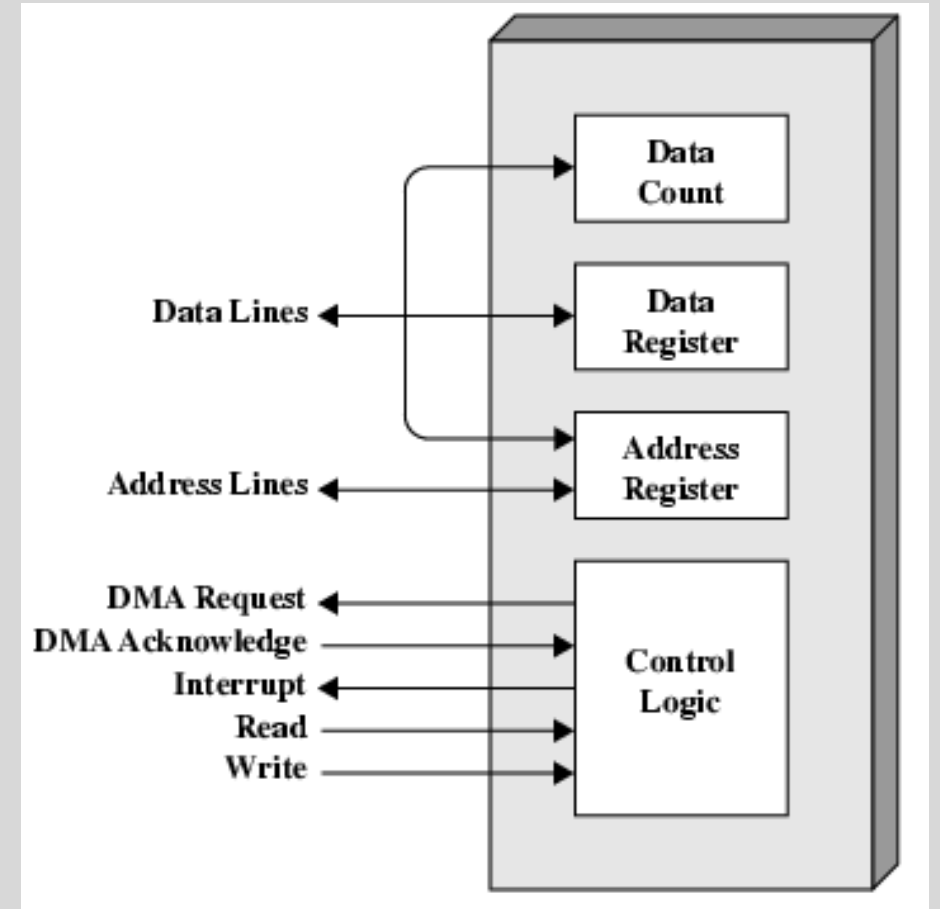
vector number	description
0	divide error
1	debug exception
2	null interrupt
3	breakpoint
4	INTO-detected overflow
5	bound range exception
6	invalid opcode
7	device not available
8	double fault
9	coprocessor segment overrun (reserved)
10	invalid task state segment
11	segment not present
12	stack fault
13	general protection
14	page fault
15	(Intel reserved, do not use)
16	floating-point error
17	alignment check
18	machine check
19–31	(Intel reserved, do not use)
32–255	maskable interrupts

Direct Memory Access

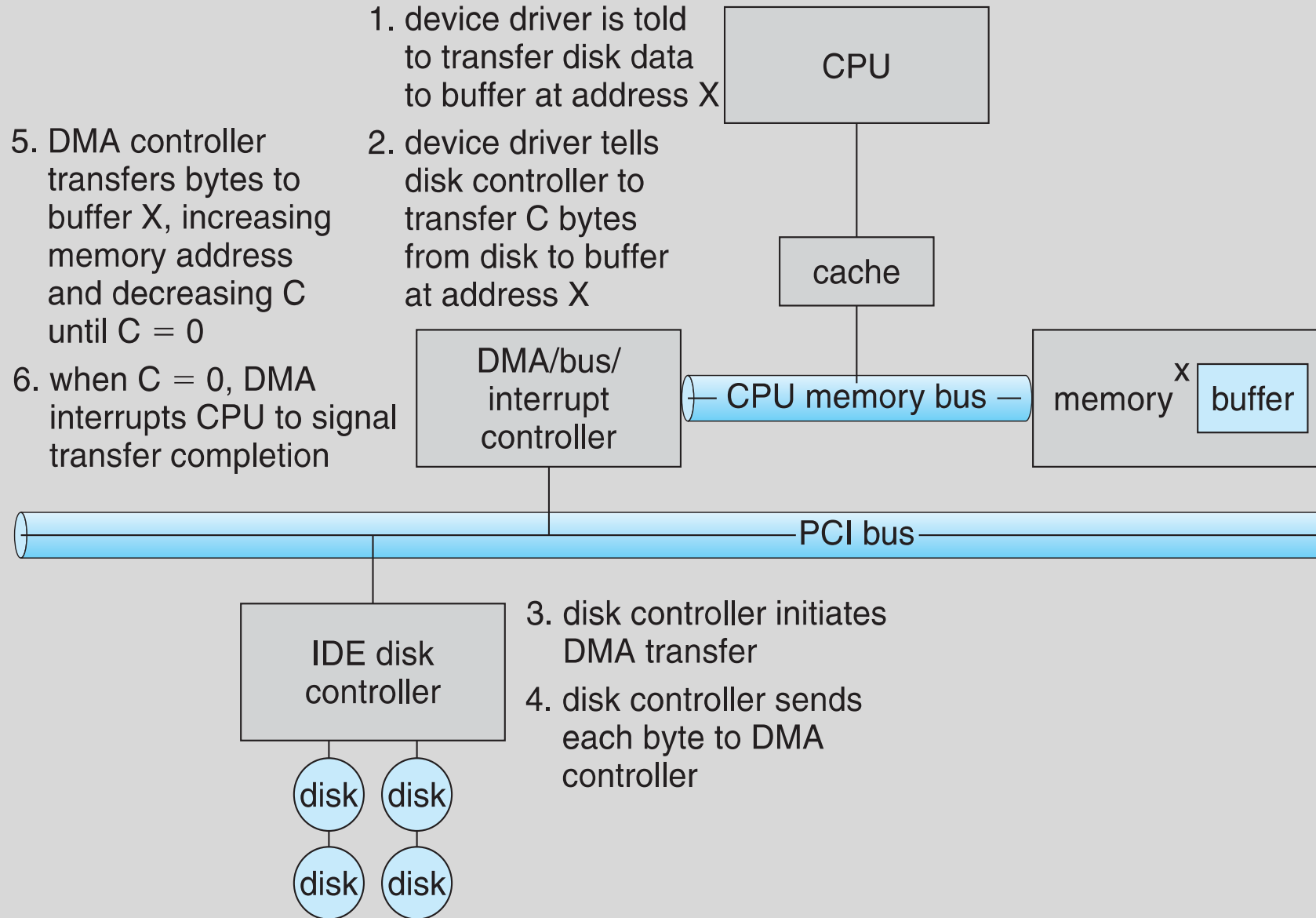
- Interrupt driven and programmed I/O require active CPU intervention
- For high-bandwidth devices (like disks) interrupt-driven I/O would consume a lot of processor cycles
- Bypasses CPU to transfer data directly between I/O device and memory
- OS writes DMA command block into memory
 - Source and destination addresses, Read or write mode
 - Count of bytes, Writes location of command block to DMA controller
- DMA is an additional module (hardware) on bus
- DMA controller takes over from CPU for I/O operations

DMA Module Diagram

- CPU tells DMA controller:-
 - Read/Write
 - Device address
 - Starting address of memory block for data
 - Amount of data to be transferred
- CPU carries on with other work
- DMA controller deals with transfer
- DMA controller sends interrupt when finished



Disk to Memory Copy via DMA



Modes of DMA operation

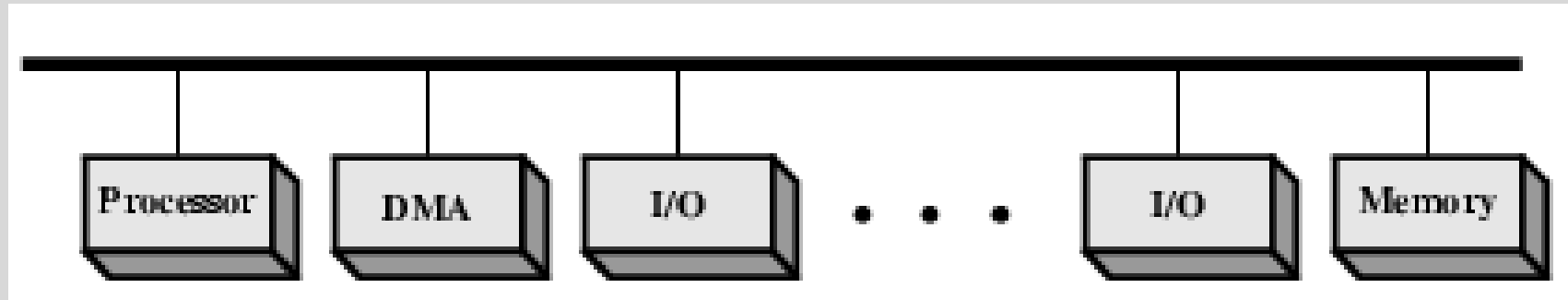
- **Cycle Stealing**

- DMA controller acquires control of bus
- Transfers a single word and releases the bus
- The CPU is slowed down due to bus contention
- Responsive but not very efficient

- **Burst Mode**

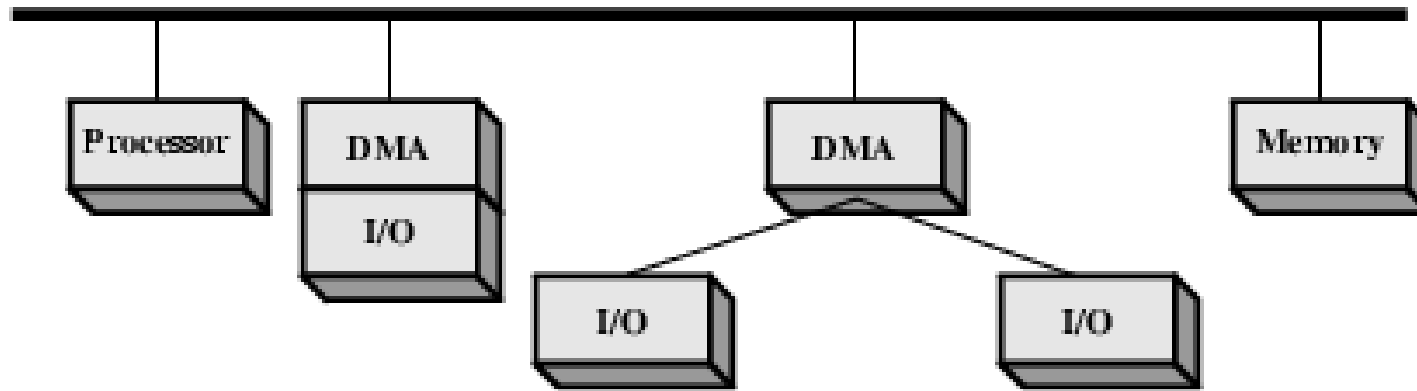
- DMA Controller acquires control of bus
- Transfers all the data and then only releases the bus
- The CPU is suspended or works with cache
- Efficient but interrupts may not be serviced in a timely way

DMA Configurations



- Single Bus, Detached DMA controller
- Each transfer uses bus twice
 - I/O to DMA then DMA to memory
- CPU is suspended twice

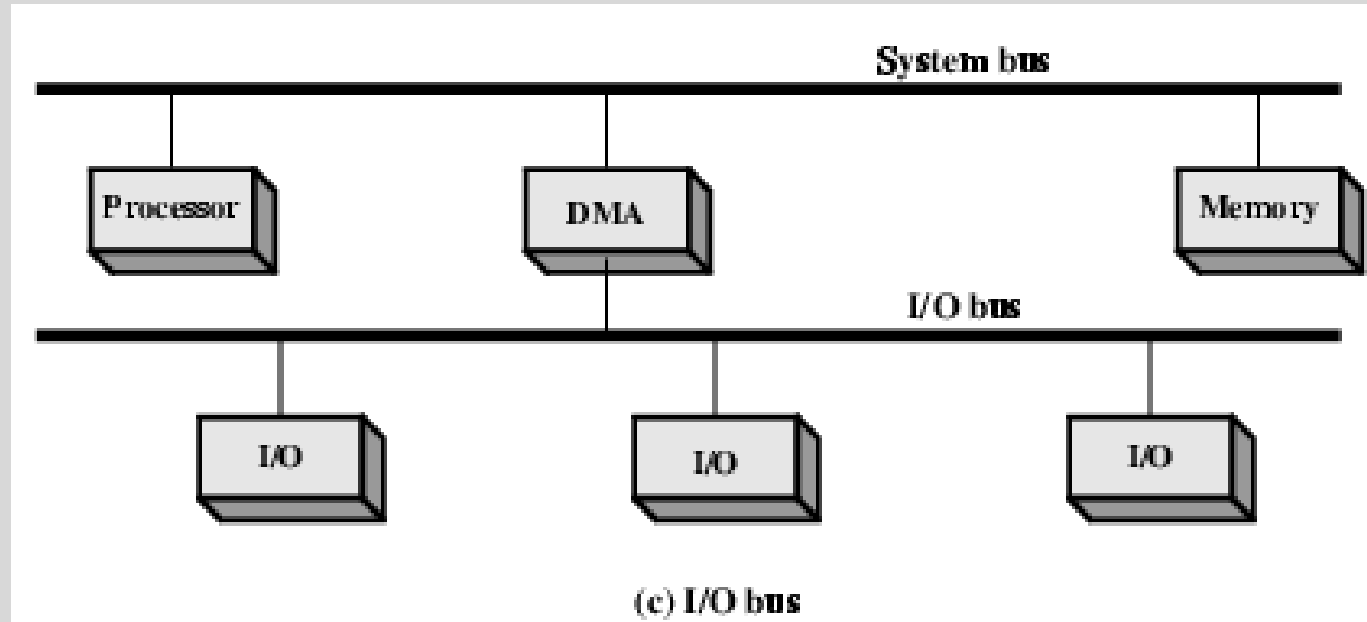
DMA Configurations



(b) Single-bus, Integrated DMA-I/O

- Single Bus, Integrated DMA controller
- Controller may support >1 device
- Each transfer uses bus once
 - DMA to memory
- CPU is suspended once

DMA Configurations



- Separate I/O Bus
- Bus supports all DMA enabled devices
- Each transfer uses bus once
 - DMA to memory
- CPU is suspended once



Thank You